

数值计算方法

带状线性方程组-直接解法

信息管理与工程学院
冯银波

三对角方程组

三对角方程组的形式如下：

$$Ax = \begin{pmatrix} d_1 & c_1 & & & \\ a_2 & d_2 & c_2 & & \\ & a_3 & d_3 & c_3 & \\ & & \ddots & \ddots & \ddots \\ & & & a_{n-1} & d_{n-1} & c_{n-1} \\ & & & & a_n & d_n \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} = b$$

该系数矩阵的带宽为3。

追赶法

将顺序 LU 分解法（普通Gauss消去法）的思路应用于三对角方程组，可得到

$$A = LU,$$

式中：

$$L = \begin{pmatrix} l_1 & & & & & \\ a_2 & l_2 & & & & \\ & a_3 & l_3 & & & \\ & & \ddots & \ddots & & \\ & & & a_{n-1} & l_{n-1} & \\ & & & & a_n & l_n \end{pmatrix}, \quad U = \begin{pmatrix} 1 & u_1 & & & & \\ & 1 & u_2 & & & \\ & & 1 & u_3 & & \\ & & & \ddots & \ddots & \\ & & & & 1 & u_{n-1} \\ & & & & & 1 \end{pmatrix}.$$

比较 $A = LU$ 两端对应的元素，得

$$d_1 = l_1, \quad c_{k-1} = l_{k-1}u_{k-1}, \quad d_k = a_k u_{k-1} + l_k, \quad k = 2, \dots, n.$$

于是

$$l_1 = d_1, \quad u_{k-1} = \frac{c_{k-1}}{l_{k-1}}, \quad l_k = d_k - a_k u_{k-1}, \quad k = 2, \dots, n.$$

追赶法求三对角方程组

得到了 $A = LU$,接下来可通过前代法+回代法求解原方程组。只是计算过程可以更加简捷,如下:

$$y_1 = b_1/l_1,$$

$$y_k = \frac{b_k - a_k y_{k-1}}{l_k}, \quad k = 2, \dots, n$$

$$x_n = y_n,$$

$$x_k = y_k - u_k x_{k+1}, \quad k = n-1, \dots, 1$$

追赶法

- 设计算法时，输入参数为：向量 a, d, c, b ，输出参数为： l, u, x
- 关于计算过程的存储问题： l, u 分别放在 d, c 的位置；前代法和回代法得到的 y, x 均存储在 b 的位置

算法：（追赶法三角分解+求解线性方程组）

$b(1) = b(1)/d(1);$

for $k = 2:n$ l_1 不需要算，依次计算 $u_1, l_2, u_2, l_3 \dots$

$c(k-1) = c(k-1)/d(k-1);$

$d(k) = d(k) - a(k) * c(k-1);$

$b(k) = (b(k) - a(k) * b(k-1))/d(k);$

end

for $k = n-1:-1:1$

$b(k) = b(k) - c(k) * b(k+1);$

end

追赶法分解定理

定理：若三对角线形方程组的系数矩阵满足

$$|d_1| > |c_1|,$$

$$|d_i| > |a_i| + |c_i|, \quad i = 2, \dots, n-1,$$

$$|d_n| > |a_n|,$$

则追赶法是可行的。

证明：满足定理条件的矩阵是严格对角占优矩阵，因此，证明过程是课后习题第8题的一个特例。

事实上，只需证 $l_k \neq 0$ 。已知 $|l_1| = |d_1| > |c_1|$ ，由归纳法，假设 $|l_{k-1}| > |c_{k-1}|$ ，则

$$\begin{aligned} |l_k| &= |d_k - a_k u_{k-1}| = \left| d_k - a_k \frac{c_{k-1}}{l_{k-1}} \right| \\ &\geq |d_k| - |a_k| \cdot \left| \frac{c_{k-1}}{l_{k-1}} \right| \geq |d_k| - |a_k| \\ &> \begin{cases} |c_k|, & k < n \\ 0, & k = n \end{cases} \end{aligned} \quad \text{证毕。}$$

高性能实现条件-提高存算效率

- 用计算机执行运算时，运行时间主要花费在：数据提取、计算、结果存储。好的算法也要注重节约花在数据存取上的时间。
- 计算器直接和寄存器交换信息。计算时，需将存储器（如磁盘）上的数据逐级传递到寄存器，再进行计算。将计算结果保存到存储器时，需将寄存器中数据逐级传递到存储器上。频繁地在寄存器和存储器之间传递数据将浪费大量的时间。
- 好的算法要节约花在数据存取上的时间，尽量将时间都用在计算上。

高性能实现条件-提高存算效率

考虑以下基本线性代数运算，比较他们的计算量、存取次数及二者之比。

典型运算	运算量	存取次数	二者之比
$y \leftarrow y + \alpha x$	$2n$	$3n + 1$	$2/3$
$y \leftarrow y + Ax$	$2n^2$	$n^2 + 3n$	2
$C \leftarrow C + AB$	$2n^3$	$4n^2$	$n/2$

可见，矩阵-矩阵运算是效率最高的。例如，做Gauss消元时，采用矩阵形式的消元比逐行进行消元效率要高很多。这样可以把时间尽量花费在计算上，而不是数据的提取上。

高性能实现条件-块状算法

对于大型矩阵，进行三角分解时，可采用分块分块三角分解。目的是减少数据的存取次数，即多使用矩阵-矩阵运算。

$$A = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1p} \\ A_{12} & A_{22} & \cdots & A_{2p} \\ \vdots & \vdots & & \vdots \\ A_{p1} & A_{p2} & \cdots & A_{pp} \end{bmatrix}$$

欲对 A 按块进行Gauss消去，最终得到分块形式的下三角阵（ L ）和上三角阵（ U ）。下面只考虑第一步Gauss消去，即以 A_{11} 为主元消去第一列。

将 A 重新看作

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}. \quad \text{这里的 } A_{12} \text{ 相当于上面的 } (A_{12}, \cdots, A_{1p})$$

欲将 A 分解成

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} L_{11} & 0 \\ L_{21} & I \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} \\ 0 & \tilde{A}_{22} \end{bmatrix}.$$

高性能实现条件-块状算法

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = \begin{bmatrix} L_{11} & 0 \\ L_{21} & I \end{bmatrix} \begin{bmatrix} U_{11} & U_{12} \\ 0 & \tilde{A}_{22} \end{bmatrix}.$$

比较上式两端可知：

$$\begin{aligned} A_{11} &= L_{11}U_{11}, & A_{12} &= L_{11}U_{12} \\ A_{21} &= L_{21}U_{11}, & A_{22} &= \tilde{A}_{22} + L_{21}U_{12} \end{aligned}$$

按下述步骤计算各个子矩阵：

- (1) 对 A_{11} 进行LU分解，得到 L_{11}, U_{11}
- (2) 解方程组 $L_{21}U_{11} = A_{21}$ 和 $L_{11}U_{12} = A_{12}$ ，得到 L_{21}, U_{12} 未知数构成一个矩阵
- (3) 计算 $\tilde{A}_{22} = A_{22} - L_{21}U_{12}$ 。

这便完成了第一列的Gauss消去，接下来可以对 $\tilde{A}_{22}$ 再进行分块，然后进行下一次Gauss消去。

矩阵-矩阵运算的高效性:以Strassen算法为例

Strassen算法: 计算矩阵相乘 $C = AB$

平分成4个 $\frac{n}{2} \times \frac{n}{2}$
的矩阵:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

计算7个中
间矩阵:

$$S_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$S_2 = (A_{21} + A_{22})B_{11}$$

$$S_3 = A_{11}(B_{12} - B_{22})$$

$$S_4 = A_{22}(B_{21} - B_{11})$$

$$S_5 = (A_{11} + A_{12})B_{22}$$

$$S_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$S_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = S_1 + S_4 - S_5 + S_7$$

$$C_{12} = S_3 + S_5$$

$$C_{21} = S_2 + S_4$$

$$C_{22} = S_1 + S_3 - S_2 + S_6$$

计算C:

以 $n = 2$ 为例,

- 传统算法需要8次乘法4次加/减法
- Strassen算法需要7次乘法18次加减法

当 n 很大时, 该方法既减少了乘法的运算量, 也减少了加减法的运算量。这体现了矩阵运算的高效性